FIT1056 – Introduction to software engineering

2026 Semester 2

PST5

The Project Journey: An Overview **- Problem-solving tasks 5**

We will use the case study of a Music School Management System (MSMS) to introduce the fundamental software engineering (SE) concepts from the perspective of a developer. Before you begin, review the **Applied#1** slides to make sure you understand how to work with the Git repository.

Music School Management System (MSMS) (Individual Task)

Objective: You will build a Music School Management System (MSMS) over five stages. Each stage is a direct upgrade of the previous one, taking you from a simple script to a robust, professional application.

Important Notes:

- This is an **individual task**.
- You are **not** required to create a formal Software Requirements Specification (SRS) document.
- You **must** use Git to track your progress. You will make a new "commit" after completing each part.

- **PST1: The Foundation.** Build a simple, in-memory prototype.
- **PST2: The Upgrade.** Add file storage, data validation, and better organization.
- **PST3: The Architecture.** Rebuild with a professional Object-Oriented (OOP) design.
- **PST4: The User Interface.** Replace the text console with a modern Graphical User Interface (GUI).
- **PST5: The Quality Assurance.** Make the application "crash-proof" and prove it works with automated tests.

Part 0: Project Setup and Git Initialization

Same Git steps of PST1 is mentioned here. You may make the changes accordingly as you want.

Goal: Create a project directory and initialize it as a Git repository. This is the foundation for your entire project.

Make sure you are on the correct branch (individual). If you have any un-committed changes, commit them now and push. Feel free to push after each individual commit, there is no problem with doing so.

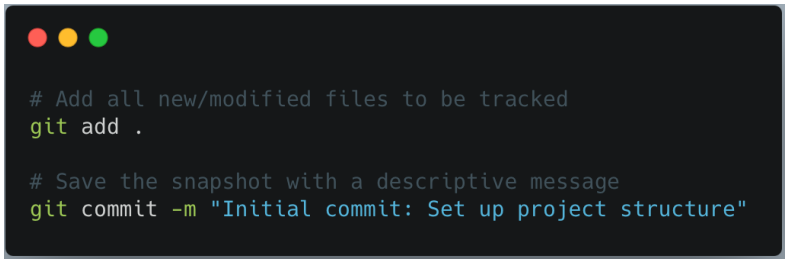
Your Task:

1. Open your terminal or command prompt (Follow the Applied-Week01 Slides).
2. Create a new folder for your project (e.g., msms-project) and navigate into it.
3. Initialize a new Git repository in this folder. This command creates a hidden .git subdirectory that will track all your changes.





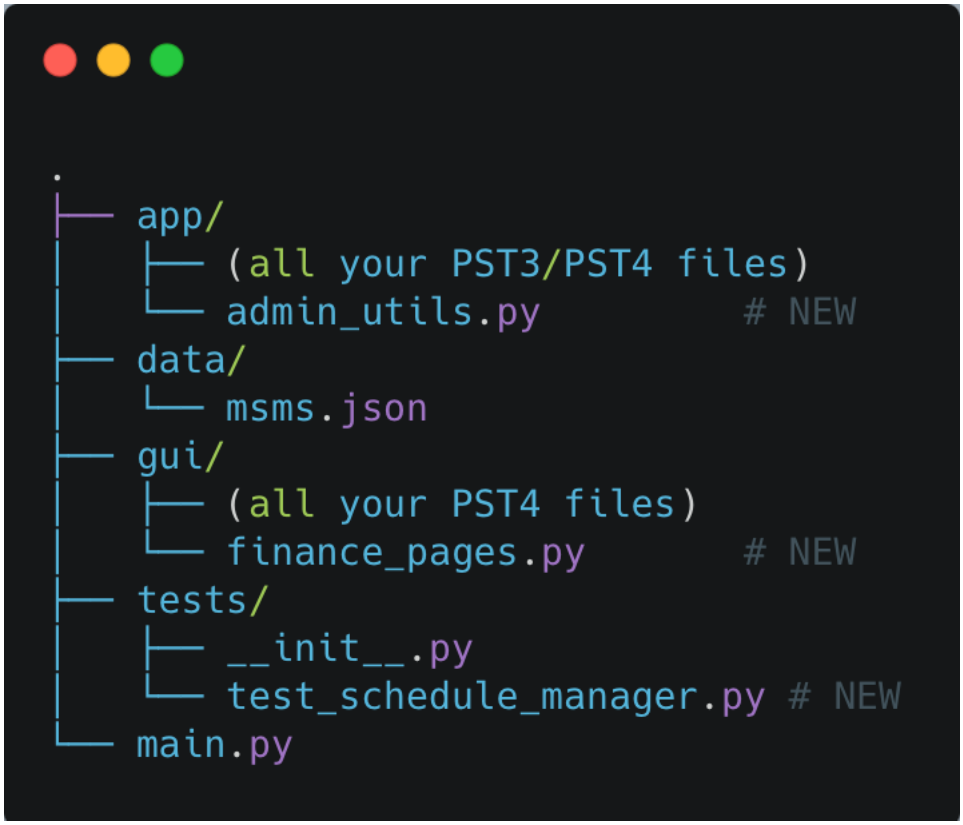
4. Create your main source code file. (e.g., main.py).
5. Add this new file to the Git staging area and make your first commit. A commit is a snapshot of your code at a specific point in time.



Part 5 (PST5): The Quality & Finance Gate

Your Goal: To solve the "Fragile & Unverified" problem. To add the final layer of professional polish by implementing critical business features (finance), adding robust administrative tools (logging, backups), and verifying the application's correctness with automated tests.

The New Directory Structure: We add the final GUI page, the admin utilities file, and the tests directory.



Fragment 5.1: The Finance & Reporting Engine

- **Mission:** To implement the core backend logic for financial transactions and exporting data reports directly within your ScheduleManager
- **Your Task:**
    1. In `app/schedule.py`, add support for a `self.finance_log` list in the `__init__`, `_load_data`, and `_save_data` methods.
    2. Create the `record_payment(student_id, amount, method)` method.
    3. Create the `get_payment_history(student_id)` method.
    4. Create the `export_report(kind, out_path)` method.

- **Check the Template in (Fragment5_1.py and add to app/schedule.py)**

Checkpoint: Commit Your Progress
Save your file and commit the change.

```
git add app/schedule.py
git commit -m "feat(finance): Implement core finance tracking and CSV reporting logic"
```

(Note: "Feat" is a common convention for a commit that introduces a new feature.)

Fragment 5.2: The GUI for Finance

- **Mission:** To expose the new financial features through the user interface, replacing the "stub" page from PST4.
- **Your Task:**
  - Create the file gui/finance_pages.py.
  - Implement the show_finance_page(manager) function.
  - This function will have two main sections: one for recording a new payment and one for viewing a student's payment history.
  - Both sections will use Streamlit forms and buttons to call the new methods on the manager object that you just created.
- **Check the Template give in:**
  - **gui/finance_pages.py**
- **Update gui/main_dashboard.py**: Import show_finance_page and replace the stub for the "Payments" page.

Checkpoint: Commit Your Progress
Save your file and commit your new function.

```
git add gui/finance_pages.py gui/main_dashboard.py
git commit -m "feat(gui): Build finance page for recording payments and viewing history"
```

Fragment 5.3: The Admin & Quality Utilities

- **Mission**: To create the professional administrative functions for logging and backups.

- **Your Task:**
  - Create the new file, app/admin_utils.py.
  - Implement init_logger(log_file) to set up Python's logging module.
  - Implement backup_data(data_path, backup_dir) to copy the data file with a timestamp.
- **Check the Template give in:**
  - **app/admin_utils.py**

```
git add app/admin_utils.py
git commit -m "feat(admin): Add logging and data backup utilities"
```

Fragment 5.4: Integrating Admin Utilities into the Application Lifecycle

- **Mission:** To make the application actually *use* the new logging and backup tools.
- **Your Task:**
  1. **Integrate Logging:**
     - In main.py, call init_logger() right at the start, before the GUI launches.
     - In app/schedule.py, import the logging module. In key methods like register_new_student, record_payment, and cancel_lesson, add logging.info() statements to record when these important business events occur.
  2. **Integrate Backups:**
     - In main.py, before launching the GUI, call backup_data() to ensure there is a safe copy of the data before the session starts.
     - Optionally, add a "Backup Now" button to the GUI that also calls this function.
- **Check the Template in main.py**
- **Check the Template in (Fragment5_2.py and add to app/schedule.py)**

```
git add app/schedule.py
git commit -m "controller: Fully integrate attendance_log into ScheduleManager state and persistence"
```

Fragment 5.4: The Automated Test Suite

- **Mission:** To write a suite of automated unit tests using pytest that verify the correctness of your application's core logic. This is the ultimate safety net.
- **Your Task:**
  - Install pytest: pip install pytest.
  - Create a tests/ directory and tests/test_schedule_manager.py.
  - Write test functions that follow the "Arrange, Act, Assert" pattern. Each function should test one specific piece of logic.
- **Check the Template give in:**
  - **tests/test_schedule_manager.py**
- **How to Run:** In your terminal, simply run the command pytest. It will automatically discover and run your tests.

Checkpoint: Commit Your Progress
Save your file and commit this important update.

```
git add tests/
git commit -m "test(qa): Implement unit tests for core logic, completing PST5"
```

```
########################################
# 4️⃣  Push the new commit(s) up to GitHub
########################################
git push origin individual
#          |      |
#          |      └─ the branch name on GitHub
#          └──────── the remote repository (GitHub by default)
```

Instructions

- **Template files provided:** You will find a skeleton for every task (Fragment#_#.py)

- **Finish · Run · Test:** Complete each fragment, run it locally, and confirm it works.
- **Commit & push:** Test the complete application, then commit and push.
- **One README to rule them all:** Create **one** high-quality README.md at the root of your repo that explains
    - what each part does,
    - how to run and test the full program, any design choices or assumptions you made.
- **A clear, detailed README is worth marks, treat it like part of the assignment.**